

Random Forest

(Bosques Aleatorios)

Analítica de Datos, Universidad de San Andrés

Si encuentran algún error en el documento o hay alguna duda, mandenme un mail a rodriguezr@udesa.edu.ar y lo revisamos.

1. ¿Qué es Random Forest?

Random Forest es básicamente un montón de árboles de decisión trabajando juntos. ¿Por qué? Porque un solo árbol puede ser muy caprichoso y ajustarse demasiado a los datos de entrenamiento (overfitting). Es como si un solo árbol dijera “bueno, vi estos datos y ahora me sé todo”, pero cuando llegan datos nuevos, se confunde. La idea de Random Forest es simple: en lugar de confiar en un solo árbol, construimos muchos árboles y que voten. Es como preguntarle a un montón de amigos su opinión en lugar de solo a uno. La mayoría probablemente tenga razón.

¿Por qué funciona mejor? Porque cada árbol ve los datos de manera diferente (usando subconjuntos aleatorios), entonces cuando se equivocan, se equivocan de maneras distintas. Al promediar sus respuestas, los errores se cancelan y obtenemos una predicción más estable.

2. ¿Cómo funciona el algoritmo?

El algoritmo de Random Forest es bastante directo:

1. **Bootstrap:** Para cada árbol, tomamos una muestra aleatoria de nuestros datos (con reemplazo). Esto significa que algunos datos podrían aparecer varias veces y otros no aparecer en absoluto.
2. **Selección aleatoria de variables:** En cada división del árbol, solo consideramos un subconjunto aleatorio de las variables disponibles. Esto evita que una variable muy importante domine todos los árboles.
3. **Construcción del árbol:** Construimos el árbol completo usando CART con los datos y variables seleccionadas.

4. **Repetir:** Hacemos esto muchas veces (típicamente 100-1000 árboles).
5. **Votación:** Para predecir, cada árbol vota y tomamos la decisión de la mayoría (clasificación) o el promedio (regresión).

3. Ventajas de Random Forest

Random Forest tiene varias ventajas que lo hacen muy popular:

- **Reduce el overfitting:** Al promediar muchos árboles, suavizamos las decisiones y evitamos ajustarnos demasiado a los datos de entrenamiento.
- **Maneja bien diferentes tipos de variables:** No le importa si son numéricas o categóricas, las maneja todas bien.
- **Es robusto:** Los outliers y el ruido en los datos no lo afectan tanto como a otros algoritmos.
- **Importancia de variables:** Nos dice qué variables son más importantes para hacer predicciones.
- **No necesita preprocesamiento:** No tenemos que normalizar los datos ni hacer transformaciones complicadas.
- **Es rápido:** Aunque construye muchos árboles, cada uno es simple y se puede hacer en paralelo.

4. Parámetros importantes

Cuando usamos Random Forest, hay algunos parámetros que podemos ajustar:

- **n_estimators:** Cuántos árboles queremos (más árboles = mejor rendimiento pero más lento). Típicamente entre 100-1000.
- **max_features:** Cuántas variables considerar en cada división. Por defecto es $\sqrt{\text{número de variables}}$ para clasificación.

- **max_depth:** Profundidad máxima de cada árbol. Si no la ponemos, los árboles crecen hasta que no se pueden dividir más.
- **min_samples_split:** Mínimo de muestras necesarias para dividir un nodo.
- **min_samples_leaf:** Mínimo de muestras en cada hoja final.
- **bootstrap:** Si queremos usar bootstrap (por defecto es True).

¿Cómo elegimos estos parámetros? La mayoría de las veces, los valores por defecto funcionan bien. Si queremos optimizar, podemos usar validación cruzada para encontrar la mejor combinación.

5. Implementación en Python

```

1 # Importamos las librerías necesarias
2 from sklearn.ensemble import RandomForestClassifier
3 from sklearn.model_selection import train_test_split
4 from sklearn.metrics import accuracy_score,
   classification_report
5 from sklearn.datasets import load_wine
6
7 # Cargamos el dataset Wine
8 wine = load_wine()
9 X = wine.data
10 y = wine.target
11
12 # Dividimos en entrenamiento y prueba
13 X_train, X_test, y_train, y_test = train_test_split(
14     X, y, test_size=0.2, random_state=42
15 )
16
17 # Creamos el modelo de Random Forest
18 rf_model = RandomForestClassifier(
19     n_estimators=100,          # 100 arboles
20     max_features='sqrt',      # sqrt del numero de variables
21     random_state=42
22 )
23
24 # Entrenamos el modelo
25 rf_model.fit(X_train, y_train)

```

```

26
27 # Hacemos predicciones
28 y_pred = rf_model.predict(X_test)
29
30 # Evaluamos el rendimiento
31 accuracy = accuracy_score(y_test, y_pred)
32 print(f'Precision: {accuracy:.3f}')
33
34 # Vemos el reporte detallado
35 print(classification_report(y_test, y_pred))

```

6. Importancia de variables

Una de las cosas más útiles de Random Forest es que nos dice qué variables son más importantes:

```

1 # Obtenemos la importancia de las variables
2 importances = rf_model.feature_importances_
3 feature_names = wine.feature_names
4
5 # Creamos un DataFrame para ver mejor
6 import pandas as pd
7 importance_df = pd.DataFrame({
8     'Variable': feature_names,
9     'Importancia': importances
10 }).sort_values('Importancia', ascending=False)
11
12 print(importance_df)
13
14 # Graficamos la importancia
15 import matplotlib.pyplot as plt
16 plt.figure(figsize=(10, 6))
17 plt.barh(importance_df['Variable'], importance_df['
    Importancia'])
18 plt.xlabel('Importancia')
19 plt.title('Importancia de Variables en Random Forest')
20 plt.show()

```

7. Comparación con CART

¿Por qué Random Forest es mejor que un solo árbol CART?

Característica	CART	Random Forest
Overfitting	Propenso	Menos propenso
Estabilidad	Baja	Alta
Interpretabilidad	Alta	Media
Velocidad	Rápido	Más lento
Precisión	Variable	Generalmente mejor

¿Cuándo usar cada uno? Si necesitamos entender exactamente cómo toma las decisiones, CART es mejor. Si queremos la mejor precisión posible y no nos importa tanto la interpretabilidad, Random Forest es la opción.

8. Notebook Anexo

En el repositorio hay un notebook anexo en el que usamos el dataset **Wine** de scikit-learn, que contiene información química de 178 vinos de tres variedades diferentes. Cada observación tiene 13 variables numéricas (alcohol, magnesio, fenoles, color, etc.) y una variable objetivo que indica la clase de vino (0, 1 o 2).

Para detalles completos de las variables y análisis exploratorio, consultar el notebook. En el experimento:

- **CART:** Se equivocó en 2 casos de 36 (94.4 % de precisión)
- **Random Forest:** Se equivocó en 0 casos de 36 (100 % de precisión)

¿Por qué Random Forest fue mejor? Porque al promediar las decisiones de 100 árboles, suavizamos los errores individuales y obtenemos una predicción más robusta. Es como tener 100 expertos en vino en lugar de uno solo.

¿Qué pasa si aumentamos el test size? Probablemente Random Forest siga siendo mejor, pero la diferencia podría ser menor. La clave está en que Random Forest generaliza mejor a datos nuevos.